3회차 · CDC / Iceberg 프로젝트 스터디

Flink에서 Iceberg로 변경 데이터를 저장하는 설계

Flink → Bronze 적재 심화
개인 도메인·테이블 설계 확정

도입 이유 · 내부 동작 · 운영 판단 · 개인 프로젝트 설계

오늘의 진행 · 권장 215분

시간	진행	남길 결과
00:00-00:50	문제·정의·대안	필요한 구성 판단
00:50-01:25	입력 SQL·복구·휴식	Bronze 입력 계약
01:25-02:20	내부 저장·실제 관찰	파일과 commit 구분
02:20-03:35	운영·휴식·개인 설계	테이블·검증 조건 확정

복습에서 확인한 것

해본 것

어디까지 실행하거나
읽어보았나요?

확인한 근거

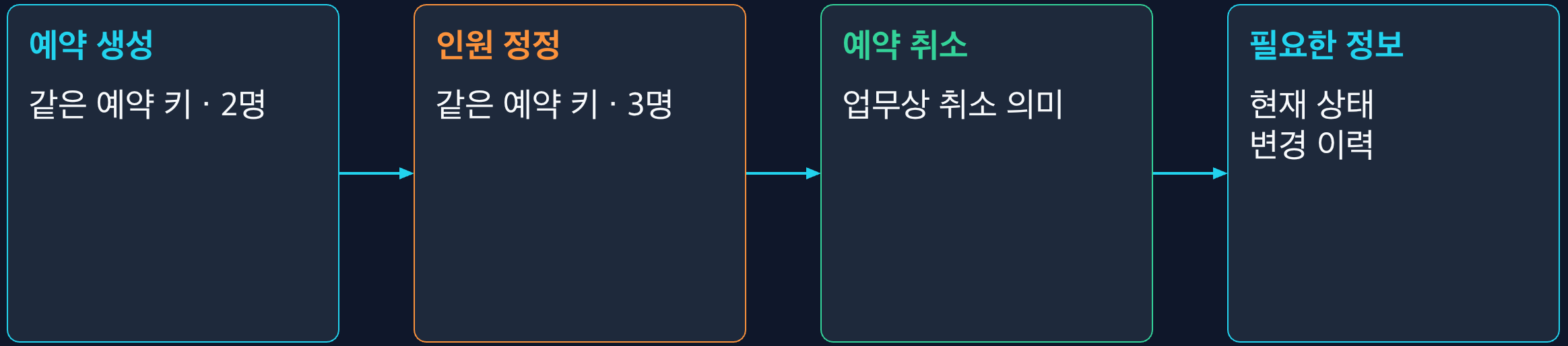
어떤 키·SQL·결과로
확인했나요?

남은 질문

어느 구간이
아직 설명되지 않나요?

실행 여부와 결과를 확인한 근거를 함께 이야기합니다

예약이 정정되고 취소된다면



가상 업무 사례 · 현재 상태만 필요한지, 중간 이력도 필요한지 구분합니다

수집 지연과 지표 지연은 다릅니다

범위	끝을 확인할 증거
원천 변경부터 Bronze	같은 키의 변경 이력이 읽힘
원천 변경부터 최종 지표	변경을 반영한 기대 집계가 읽힘

Bronze가 새로워져도 Silver·Gold는 별도 갱신이 필요합니다

왜 CDC와 Kafka를 연결할까요?

CDC

전체 재조회 대신
변경 이력을 전달

Kafka

입력을 보관하고
소비 속도를 분리

원천 부하·WAL·브로커 보관 비용도 함께 설계합니다

Flink는 무엇을 실행하나요?

계속 처리

source에서 읽고
연산 후 sink로 전달

복구 상태

입력 위치와 처리 상태를
checkpoint로 보존

현재 수업

Kafka 원문을 읽어
Bronze에 append

Flink는 실행 엔진이며, 영속 테이블의 파일 목록은 Iceberg가 관리합니다

Flink의 실행과 상태는 어디에 있나요?



Iceberg는 무엇을 관리하나요?

테이블 상태

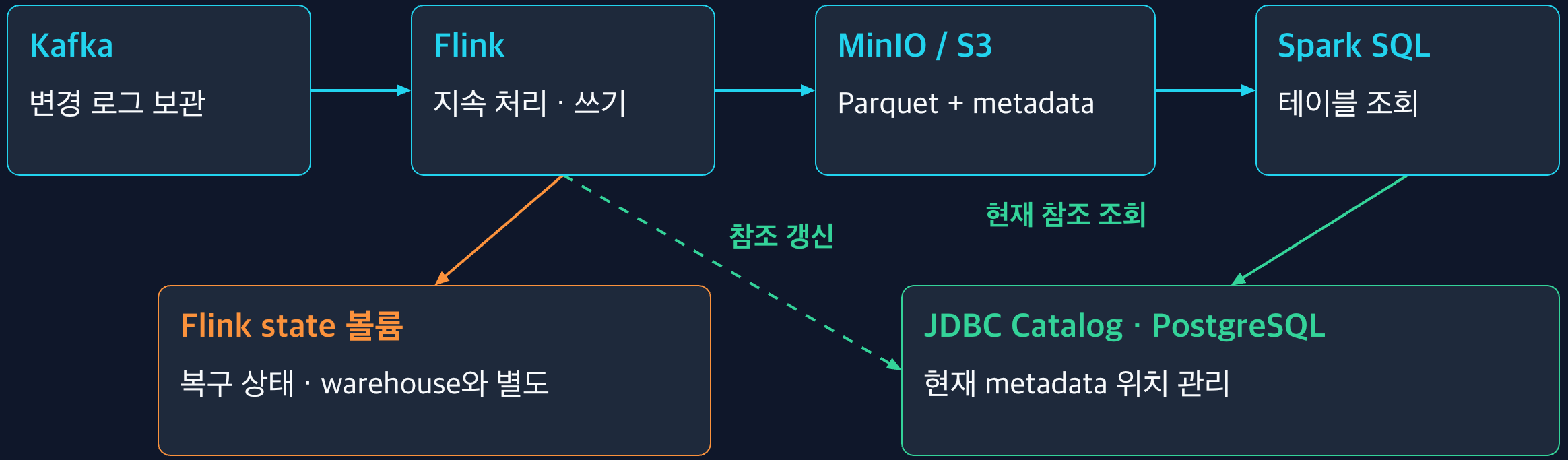
스키마와 snapshot
유효한 파일 목록

쓰기와 읽기

여러 엔진이 같은
테이블 규칙을 사용

저장소·쿼리 엔진·테이블 형식의 역할을 나눠 봅니다

엔진·저장소·Catalog의 역할



Iceberg: 이 파일·스키마·snapshot을 테이블로 관리하는 규칙

Bronze는 지속 적재 · Silver·Gold는 실행 시 갱신 · Airflow는 선택 구성

Parquet 파일과 Iceberg 테이블

구분	Parquet	Iceberg
관리 단위	컬럼 데이터 파일	파일들을 묶은 테이블 상태
저장 내용	실제 행·컬럼 통계	스키마·snapshot·파일 참조
읽기 범위	지정한 파일의 데이터	선택한 snapshot의 유효 파일

현재 랩은 Parquet를 데이터 파일로 사용하는 Iceberg 테이블입니다

수집·현재 상태·지표의 책임

Bronze

원문·전달 위치
변경 이력 보존

Silver

키·최신 판단·삭제
현재 상태 구성

Gold

업무 정의에 따른
집계와 검증

같은 업무 키의 이력 수와 현재 건수를 구분합니다

같은 요구를 다른 구성으로 풀 수 있습니다

선택지	검토할 조건	함께 드는 비용
Airflow + Spark 배치	시간·일 단위 갱신이면 충분	변경 발견·재조회 범위
Spark Structured Streaming	Spark 기반 마이크로배치 활용	trigger·상태·파일 관리
Kafka Connect Iceberg Sink	전달·routing 기능이 요구에 맞음	connector·commit 설정
Flink + Iceberg	연속 처리·상태·복구가 필요	상시 엔진과 상태 운영

필요한 지연과 처리 규칙으로 비교합니다 · 이 수업의 벤치마크 결과는 아닙니다

내 요구에서 필요한 구성을 고르기

업무 조건

현재 값·중간 이력
허용 지연·보존 기간

구성 선택

선택한 경로와 이유
비교할 대안 하나

학습지: 업무 질문·두 지연 목표·선택 이유를 잠정 기록합니다

Flink는 Kafka value를 원문으로 읽습니다

선언	실제 의미
payload STRING	Kafka value를 받을 한 컬럼
format = raw	업무 JSON을 해석하지 않음
topic-pattern	orders·payments·refunds 입력
INSERT SELECT	원문과 전달 정보를 Bronze에 저장

원문 보존과 JSON 유효성 검증은 별도 책임입니다

Kafka 전달 위치도 함께 저장합니다

입력 정보·계산식	Bronze 컬럼	읽을 의미
topic / partition	topic / partition_id	어느 입력 구간인가
offset	kafka_offset	그 파티션의 전달 위치
timestamp	kafka_ts	Kafka 레코드 시각
CURRENT_TIMESTAMP	ingested_at	Flink 처리 시각

VIRTUAL로 읽은 값도 SELECT로 투영하면 Bronze에 저장됩니다

삭제 JSON과 tombstone은 어떻게 남나요?

op=d JSON

본문이 있는 삭제 이력
Bronze에 append

wire-null tombstone

같은 키의 null value
현재 필터에서 제외

payload IS NOT NULL

본문 존재 여부 필터
업무 삭제와 구분

Bronze의 op=d 행은 Iceberg DeleteFile이 아닙니다

append 이력에서 현재 상태를 읽으려면

Bronze append

생성·정정·삭제를
각각의 행으로 보존

현재 상태 해석

같은 키의 최신 판단
삭제·반복 전달 규칙 적용

이력 세 행이 현재 환불 세 건을 의미하지는 않습니다

checkpoint는 무엇을 남기나요?

복구 지점

입력 위치와 연산 상태를
일관되게 보존

현재 경로

flink-state 볼륨
checkpoint·savepoint

10초 간격

주기 설정이며
조회 지연 상한은 아님

Flink 상태 저장과 Iceberg 테이블 확정을 나누어 확인합니다

earliest 시작과 저장 상태 복원

새 시작

저장 상태가 없으면
startup 설정을 적용

상태 복원

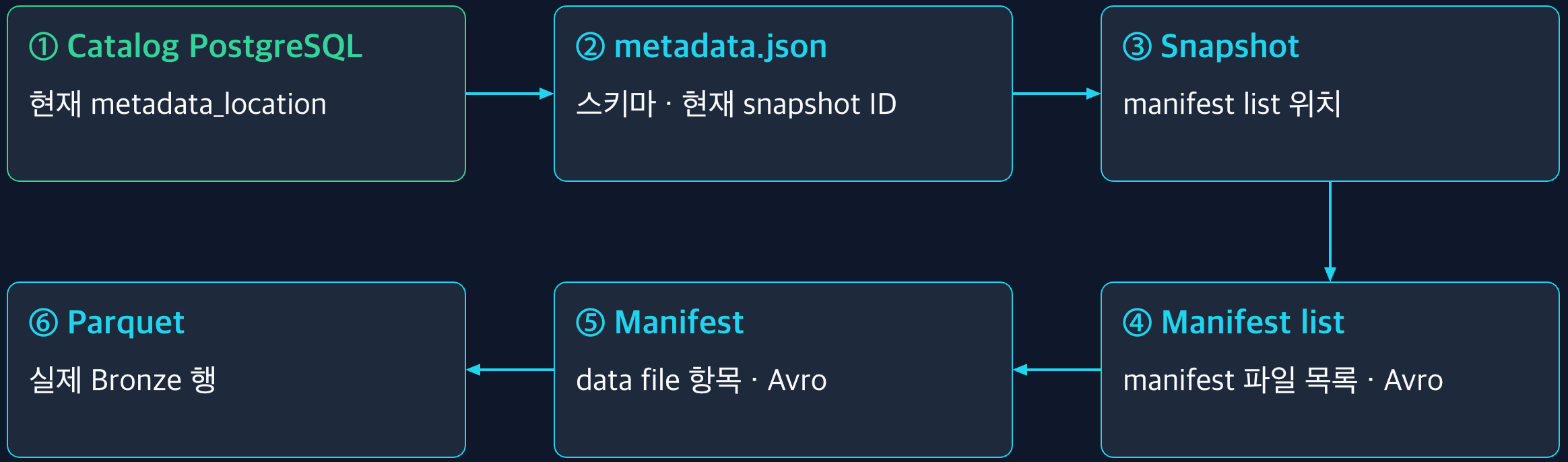
저장된 입력 위치와
pending 작업을 이어감

상태 없이 재제출하는 것과 복구하는 것을 구분합니다

10분 쉬고 이어가겠습니다

돌아오면 파일 준비부터 테이블 commit까지 따라갑니다

Iceberg reader가 파일을 찾는 순서



metadata.json · manifest list · manifest · Parquet는 MinIO / S3에 저장

Catalog의 현재 참조가 읽을 metadata와 snapshot을 결정합니다

snapshot은 테이블 전체 복사일까요?

새 테이블 상태

새 snapshot과
파일 참조를 확정

기존 파일 재사용

바뀌지 않은 데이터는
여러 snapshot에서 공유

snapshot 수와 데이터 전체 복사본 수는 같지 않습니다

레코드가 오면 writer는 파일을 작성합니다

계속 들어오는 행

Kafka → filter → writer

Writer의 파일 준비

버퍼·파일 쓰기는 계속 진행

Checkpoint barrier

열린 파일 마무리 · WriteResult

checkpoint 간격 전체를 메모리에만 모았다가 처음 쓰는 구조가 아닙니다

Pending 파일 참조를 checkpoint 상태에 보관

아직 테이블에 commit되지 않은 준비 파일일 수 있습니다

연속 파일 작성 → barrier 전 파일 마감 → pending 결과를 복구 상태에 보존

checkpoint 완료 뒤 테이블을 commit합니다

① Checkpoint 완료 통지

복구 상태가 저장됨

② Iceberg committer

준비된 파일·metadata commit

③ Catalog 참조 갱신

현재 metadata 위치 원자적 변경

파일 준비와 테이블 공개는 다른 단계 · 파일 전체가 한꺼번에 업로드되는 트랜잭션이 아닙니다

④ 새로 실행한 Spark SQL 조회

현재 snapshot의 manifest를 따라 포함된 파일을 읽습니다

파일과 metadata 준비 후, 한 테이블의 현재 metadata 참조를 원자적으로 갱신합니다

checkpoint가 성공해도 commit은 따로 봅니다

Flink checkpoint

입력과 처리 상태의
복구 지점이 완료

Iceberg commit

새 파일을 포함한
테이블 상태가 확정

키별 SELECT

내가 찾는 변경이
지금 조회되는지 확인

completed checkpoint · 성공 commit · 업무 키 조회를 별도로 확인합니다

exactly-once의 경계

sink 복구

복원 상태와 commit 기록으로
같은 결과의 재commit을 판단

업무 중복

다른 전달 위치의 반복은
별도 키·버전 규칙이 필요

EXACTLY_ONCE 설정과 전체 업무 중복 제거를 같은 보장으로 읽지 않습니다

관찰 A · 원천 0행과 Bronze 3이력

원천 현재

환불 행 0개
삭제된 현재 상태

Bronze 이력

c → u → d, 총 3개
변경 순서와 전후 값

기존 완료 키 읽기 전용 · 선택한 키와 실제 조회 결과를 대조합니다

관찰 B · MinIO 콘솔에서 확인하기

Catalog 참조

현재 JSON 위치 확인

버킷·prefix

warehouse
bronze/cdc_events/

파일 관계

JSON · Avro · Parquet

기대 기본 경로와 실제 catalog 참조를 구분해 확인합니다

data 파일과 .files 결과를 연결합니다

표시 항목	확인할 의미
file_path	현재 snapshot에 포함된 파일 경로
file_format	이 파일의 데이터 형식
record_count	파일 전체 행 수
file_size_in_bytes	파일 크기 · byte 단위

.files는 metadata 조회 · 물리 파일 디코드와 특정 키 행 수는 별도 확인합니다

현재 metadata JSON은 어떻게 고를까요?

현재 참조

catalog-db의
metadata_location

SQL에서 대조

metadata 변경 이력
snapshot ID·시각·operation

버킷의 최신 파일과 Catalog의 현재 참조는 같은 판단 기준이 아닙니다

같은 키의 행과 Parquet 경로

키 조건

payload JSON의 before·after에서
같은 업무 키 추출·검색

파일 연결

_file AS parquet_file
현재 .files와 경로 대조

DELETE는 before의 키도 확인합니다 · _file 경로가 bytes 디코드를 대신하지는 않습니다

관찰 C · 새 변경 한 건의 가시성



관찰하지 못한 중간 상태는 추정해 채우지 않습니다

지연을 잴 때 시작과 끝을 정합니다

측정 이름	시작	끝
원천부터 Bronze	확보한 원천 commit 기준	같은 변경이 읽힌 시점
wait 관찰 시간	wait 명령 시작	첫 Bronze 관측
원천부터 지표	같은 원천 commit 기준	집계 기대값이 읽힌 시점

목표 지연·실측 지연·관찰 시작부터의 시간을 나누어 적습니다

짧은 checkpoint와 작은 파일

짧은 간격

잦은 commit 기회
flush·metadata 비용 증가 가능

긴 간격

빈도 비용을 줄일 여지
가시성 대기·복구 범위 증가 가능

checkpoint 주기·유입량·writer 수·실제 파일 크기를 함께 봅니다

파일 정리 작업도 목적이 다릅니다

작업	목적	주의할 것
Compaction	작은 data 파일 재작성	쓰기·연산 비용과 동시 작업
Snapshot expiration	과거 table 버전 보존 종료	조회·복구에 필요한 버전
Orphan 정리	참조되지 않는 객체 정리	미커밋 파일·안전 보존 시간

버킷에서 오래된 파일부터 직접 지우는 방식으로 대체하지 않습니다

운영에서 볼 최소 증거

범위	볼 증거	답하는 질문
입력	rate · pending / lag	읽지 못한 입력이 쌓이나
상태	checkpoint 성공·실패·시간	복구 지점이 진행하나
commit	마지막 성공 경과·소요 시간	table 확정이 진행하나
결과	snapshot · 같은 키 SELECT	필요한 변경이 읽히나

RUNNING 상태에 입력·commit·결과 증거를 더합니다

Backpressure는 어디서 시작됐을까요?

처리 경로

느린 하류가
상류 전송을 막을 수 있음

조사 순서

subtask 시간·I/O·barrier
지연이 시작된 경계 확인

원인을 확인한 뒤 병렬도·저장소·상태 비용을 판단합니다

사례 · RUNNING인데 데이터가 오래됐습니다

먼저

새 변경·대상 키·조회 조건

다음

Kafka · 처리 · commit
같은 키 SELECT

마지막으로 확인된 경계와 아직 모르는 경계를 구분합니다

사례 · checkpoint는 성공, commit은 오래됐습니다

관측

입력·checkpoint 진행
마지막 commit은 오래됨

가설 확인

committer 로그·Catalog·저장소
현재 snapshot·같은 키 SELECT

commit 경로를 조사할 근거이며, 원인을 확정한 결과는 아닙니다

복구 시간만큼 입력도 남아 있어야 합니다

Kafka와 상태

복원할 입력 범위
checkpoint·savepoint 보존

원천과 CDC

slot·WAL의 진행
원천 디스크 보호

입력·상태·테이블 파일 중 하나가 사라지면 복구 조건이 달라집니다

현재 랩에서 검증한 범위를 구분합니다

현재 구성

단일 노드·병렬도 1
로컬 영속 상태

현재 관측

키별 결과와 순차 조회
전체 운영 보장과 구분

후속 갱신

Silver·Gold 실행 시 반영
자동 스케줄은 선택

랩의 성공 결과를 운영 규모·HA·지연 보장으로 확대하지 않습니다

5분 쉬고 개인 설계를 이어갑니다

원천 테이블·입력 계약·검증 조건을 확정합니다

원천 테이블과 변경 의미 확정

정할 것	설계에 남길 내용
테이블·관계	2~3개 · 한 행 의미 · PK/FK
시간·순서	변경 시각·버전의 기준과 한계
발생 변경	INSERT / UPDATE / DELETE 사례
삭제·후속 활용	물리·소프트 삭제 · 조인·집계 한 줄

이름만 적지 말고 한 행·키·변경 사례를 함께 정합니다

내 Bronze 입력 계약

계약 항목	내 설계에서 정할 것
입력과 식별	토픽 · 업무 키(PK) · 전달 위치 식별
보존	payload · 전달 위치 · 시각 의미
제외와 해석	tombstone · 삭제 · 잘못된 입력
복구와 접근	상태·입력 보관 · 원문 접근 범위

raw 보존을 선택한 이유와 후속 해석의 책임을 적습니다

같은 키로 결과를 검증하기

사례	예상할 결과	확인 근거
생성·수정·삭제	이력과 현재 상태를 각각 정의	같은 키·전후 값·op
반복 전달	현재 값·지표의 유지 조건	업무 키·버전 규칙
실패 후 복구	어디부터 이어갈지	남아 있는 입력·상태
지연 목표	시작과 종료의 의미	기준 시각·같은 키 SELECT

입력 사례 하나에 기대값과 조회 조건을 붙입니다

동료의 설계에 한 가지 질문하기

대표 설계 2개

8분 · 유형이 다른 사례
키·삭제·복구를 질문

전원 자가 점검

4분 · 체크리스트 대조
결정 또는 보류 질문 반영

대표 2개 공개 리뷰 8분 + 전원 자가 점검 4분

오늘 확정한 설계와 다음 Silver

오늘 확정

도메인·테이블·변경 의미
Bronze 입력과 검증 조건

다음 구현

키·순서·삭제를 해석해
Silver 현재 상태 구성

개인 실행

공통 실행 안내에서
내 환경과 범위를 선택

8회 모두 수업 · 프로젝트 발표는 별도 일정입니다

선택 부록 · 행 변경의 CoW와 MoR

Copy-on-Write

변경 대상 data file을
다시 작성

Merge-on-Read

추가 변경 정보를
읽을 때 함께 반영

엔진·연산별 지원을 확인합니다 · 현재 Bronze raw append와 구분합니다

선택 부록 · 스키마와 파티션의 변화

Schema evolution

컬럼의 field ID로
테이블 스키마 변경 관리

Partition evolution

이후 쓰기의 분할 규칙 변경
기존 파일과 함께 조회

원천 JSON 변경 · Bronze 컬럼 변경 · 후속 파싱 변경을 나눠 봅니다

선택 부록 · 이 버전의 sink와 commit 지표

구현·지표	확인할 역할
IcebergStreamWriter	레코드 작성·barrier 전 파일 마감
IcebergFilesCommitter	pending 상태·완료 통지 후 commit
elapsedSecondsSince LastSuccessfulCommit	마지막 성공 commit 이후 경과
lastCommitDurationMs	최근 commit 소요 시간

Flink 1.20 / Iceberg 1.10.0 기본 legacy sink 기준 · 실제 유효 설정은 별도 확인

공식 근거와 수업 자료

분류	공식 근거·확인할 내용
Flink 1.20	실행·checkpoint·Kafka·backpressure
Iceberg 1.10	테이블 구조·JDBC·Flink 쓰기·유지관리
고정 릴리스 소스	writer·committer·Catalog 갱신
CDC와 전달	PostgreSQL 17·Debezium 3.2·Kafka 3.9
학생 자료	labs/session03 · docs/self-study.md

공식 문서 링크 모음: labs/session03/references.md